



Inductive Predicate Synthesis Modulo Programs (IPS-MP)

Scott Wesley



Jorge A. Navas



Valentin Wüstholtz



Maria Christakis



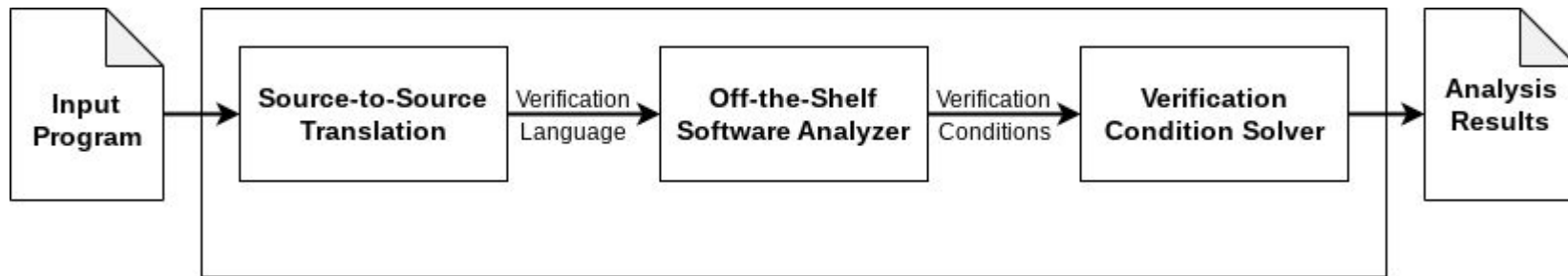
Arie Gurfinkel
Richard Treffer



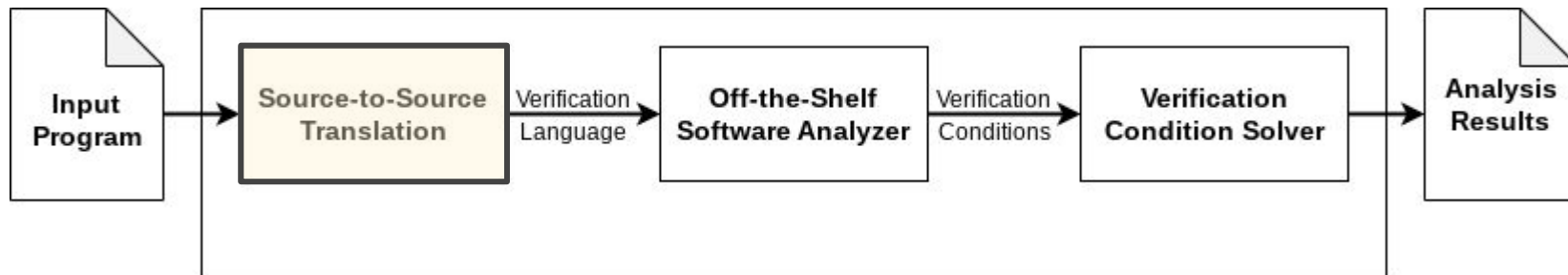
Overview and Motivation

Integrating the Solver with the Verifier

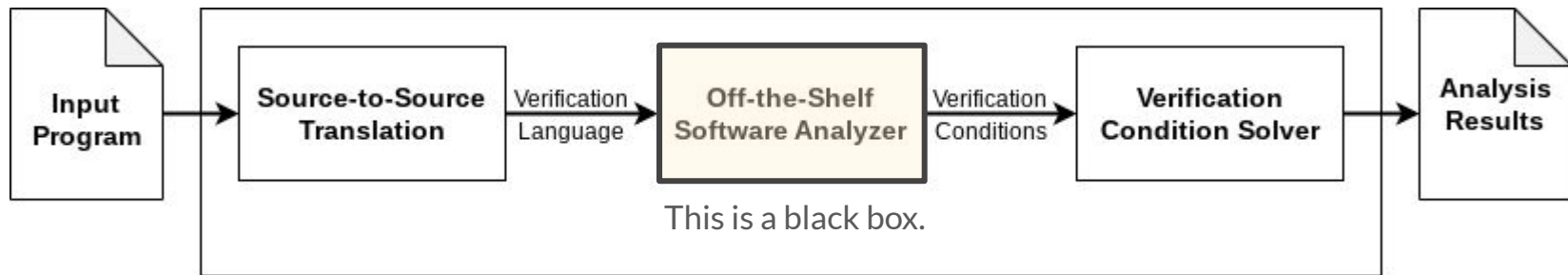
Building Tools from Off-the-Shelf Analyzers



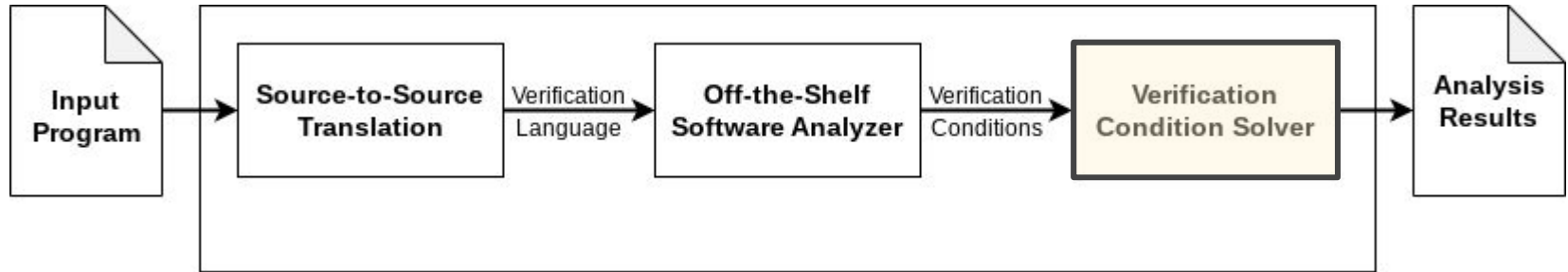
Building Tools from Off-the-Shelf Analyzers



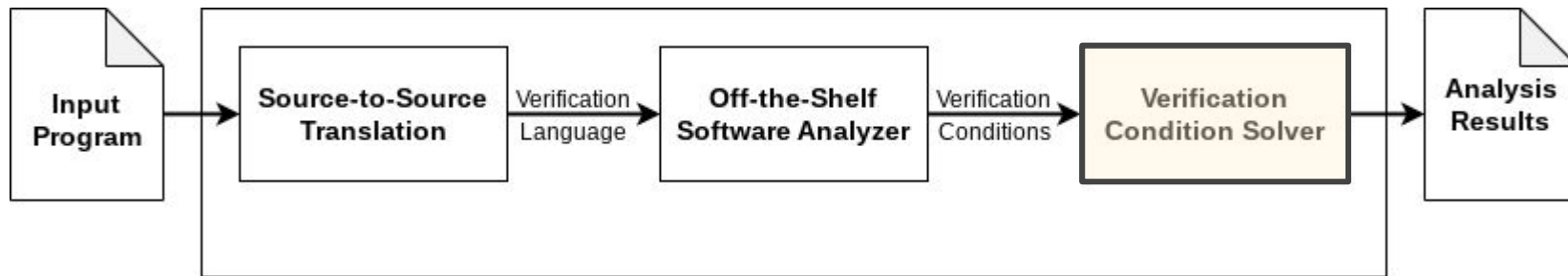
Building Tools from Off-the-Shelf Analyzers



Building Tools from Off-the-Shelf Analyzers

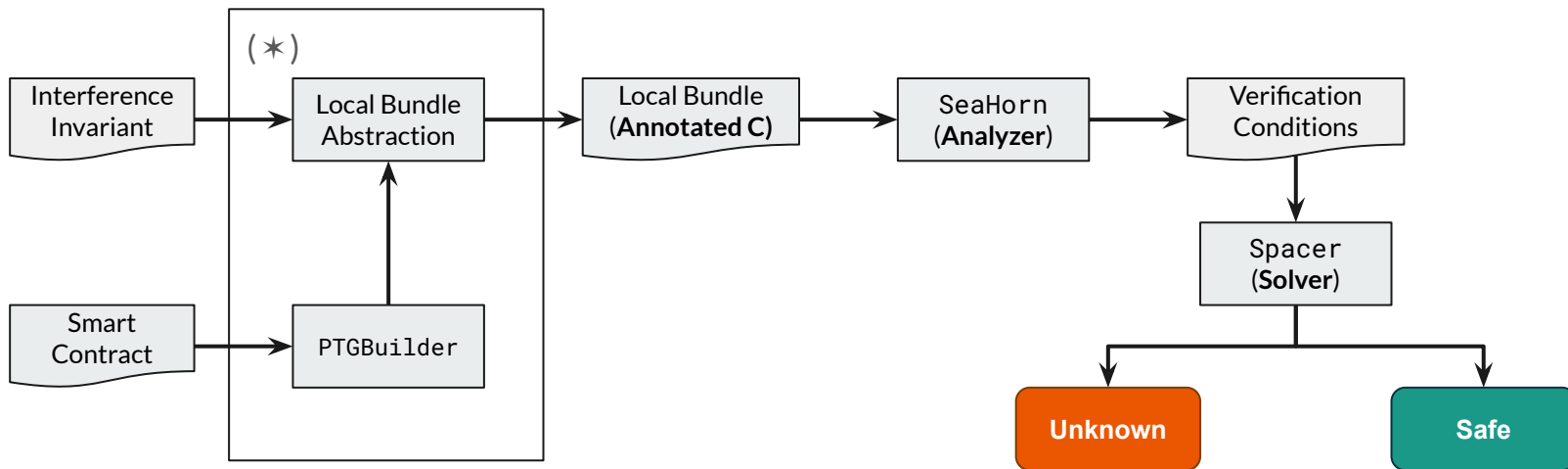


Building Tools from Off-the-Shelf Analyzers



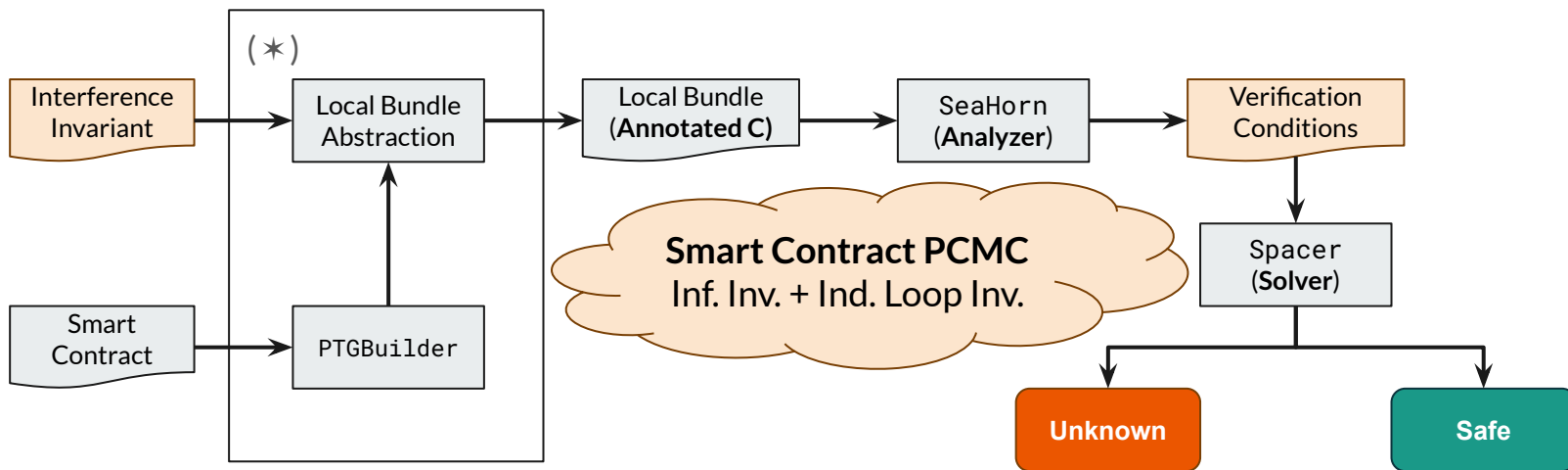
There are many well-known examples of this design pattern (i.e., using off-the-shelf analyzers to build new software verification tools). In this talk we will focus on **SmartACE**, which is built upon **SeaHorn**.

SmartACE: An Overview



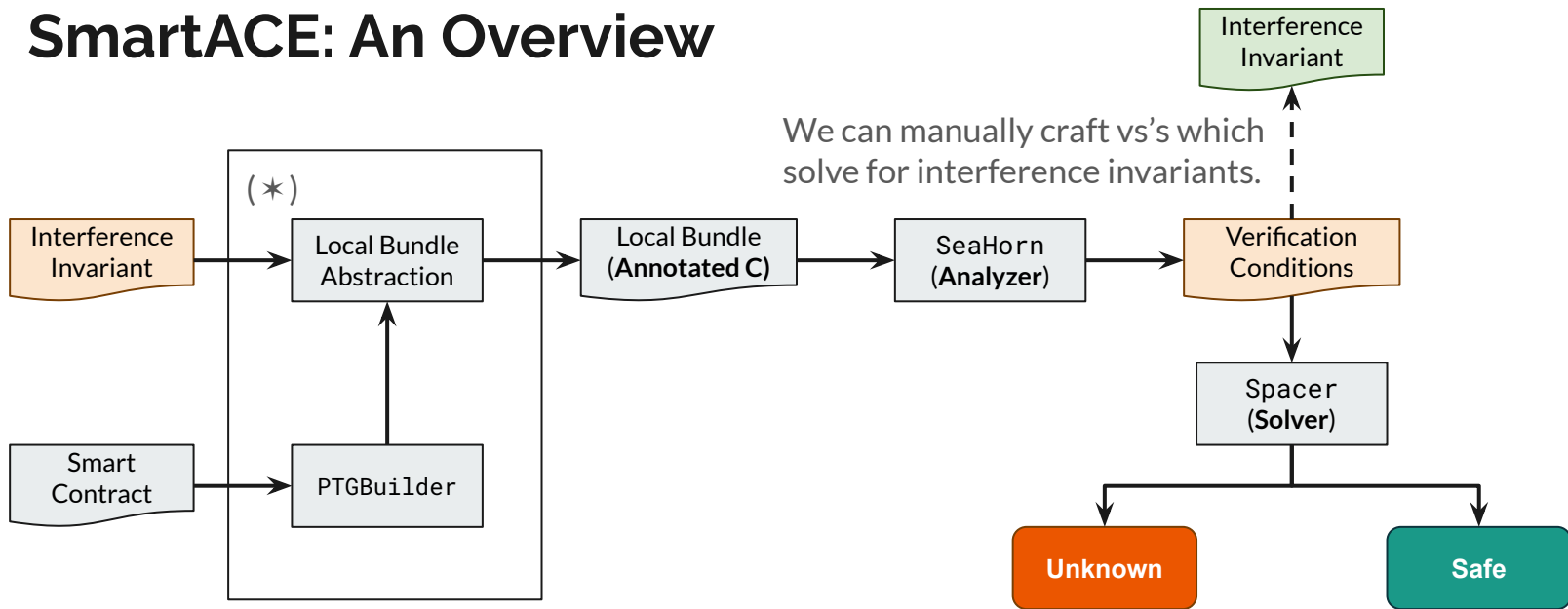
(*) *Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE (2022)*. S. Wesley, M. Christakis, J. A. Navas, R. J. Treffer, V. Wüstholtz, A. Gurfinkel. In VMCAI 2022.

SmartACE: An Overview



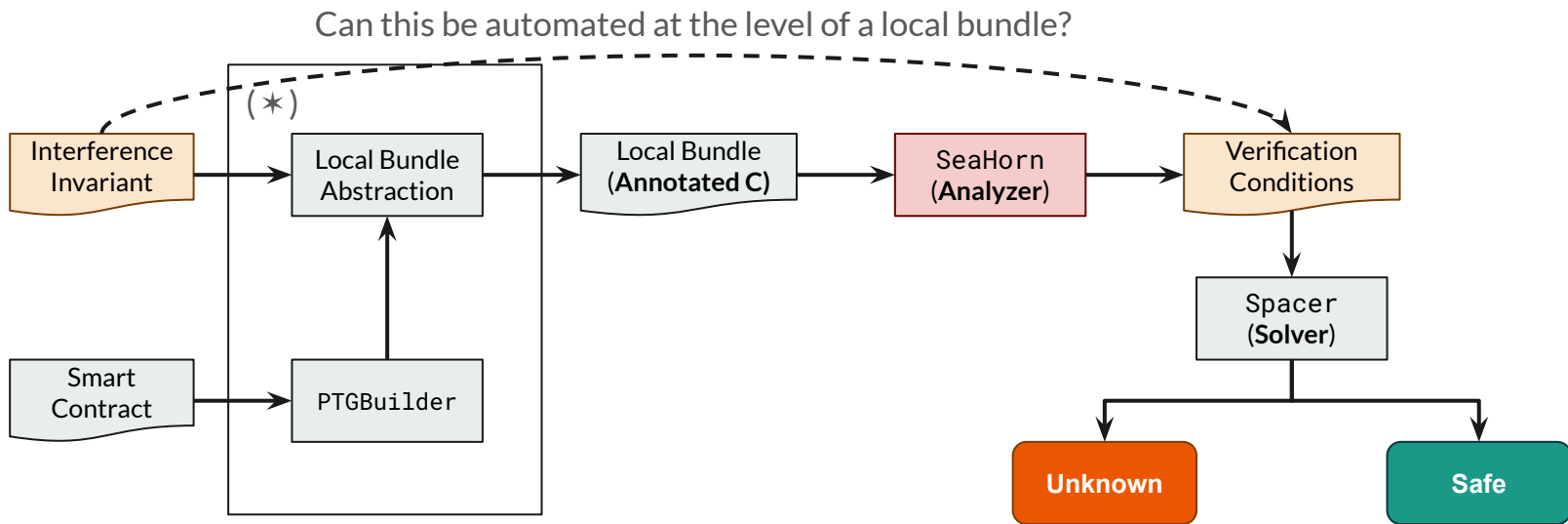
(*) *Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE* (2022). S. Wesley, M. Christakis, J. A. Navas, R. J. Trefer, V. Wüstholtz, A. Gurfinkel. In VMCAI 2022.

SmartACE: An Overview



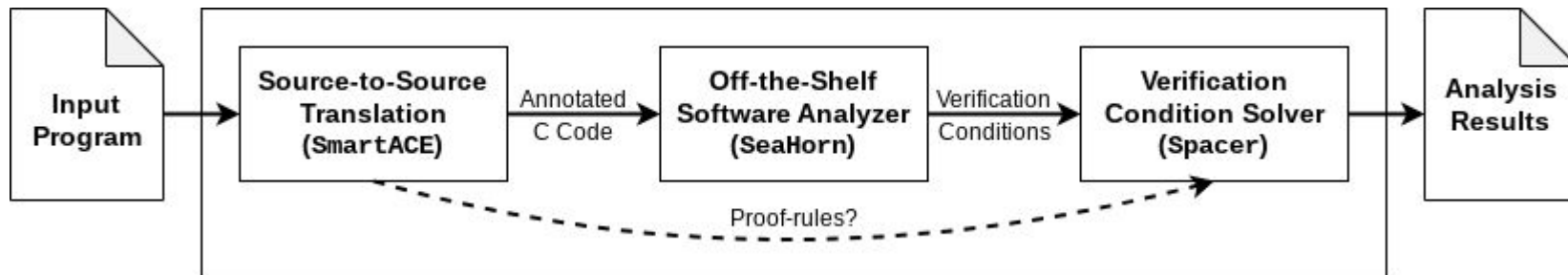
(*) *Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE* (2022). S. Wesley, M. Christakis, J. A. Navas, R. J. Trefer, V. Wüstholtz, A. Gurfinkel. In VMCAI 2022.

SmartACE: An Overview



(*) *Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE* (2022). S. Wesley, M. Christakis, J. A. Navas, R. J. Trefer, V. Wüstholtz, A. Gurfinkel. In VMCAI 2022.

There's an Impedance Mismatch





Specifying Proof Rules in Source Code

As a first step, let's see what it would look like to specify proof rules at the level of source code. As a **toy example** (to save time) we will consider the case of encoding **adequate loop invariant inference**.

```
1 void main(int x) {  
2     assume(x >= 0);  
3     int i = 0; int y = 0;  
4     while (i < x) {  
5         i += 1; y += 2;  
6     }  
7     assert(y == 2 * x);  
8 }
```

Specifying Proof Rules in Source Code

As a first step, let's see what it would look like to specify proof rules at the level of source code. As a **toy example** (to save time) we will consider the case of encoding **adequate loop invariant inference**.

```
1 void main(int x) {  
2   assume(x >= 0);  
3   int i = 0; int y = 0;  
4   while (i < x) {  
5     i += 1; y += 2;  
6   }  
7   assert(y == 2 * x);  
8 }
```

Inv is the loop
invariant

```
1 void main(int x) {  
2   assume(x >= 0);  
3   int i = 0; int y = 0;  
4   assert(Inv(x, y, i));  
5  
6   y = *; i = *;  
7   assume(Inv(x, y, i));  
8   if (i < x) { // Unrolled  
9     i += 1; y += 2;  
10    assert(Inv(x, y, i));  
11    assume(false);  
12  }  
13  
14  assert(y == 2 * x);  
15 }
```

Specifying Proof Rules in Source Code

As a first step, let's see what it would look like to specify proof rules at the level of source code. As a **toy example** (to save time) we will

Base Case: $x \geq 0 \Rightarrow \text{Inv}(x, 0, 0)$

```
1 void main(int x) {  
2   assume(x >= 0);  
3   int i = 0; int y = 0;  
4   while (i < x) {  
5     i += 1; y += 2;  
6   }  
7   assert(y == 2 * x);  
8 }
```

```
1 void main(int x) {  
2   assume(x >= 0);  
3   int i = 0; int y = 0;  
4   assert(Inv(x, y, i));  
5  
6   y = *; i = *;  
7   assume(Inv(x, y, i));  
8   if (i < x) { // Unrolled  
9     i += 1; y += 2;  
10    assert(Inv(x, y, i));  
11    assume(false);  
12  }  
13  
14  assert(y == 2 * x);  
15 }
```

Inv is the loop
invariant

Specifying Proof Rules in Source Code

As a first step, let's see what it would look like to specify proof rules at the level of source code. As a **toy example** (to save time) we will specify the proof rules for a simple loop invariant inference.

Base Case: $x \geq 0 \Rightarrow \text{Inv}(x, 0, 0)$

```
1 void main(int x) {
```

Inductive Case: $\text{Inv}(x, y, i) \wedge (i < x) \Rightarrow \text{Inv}(x, y + 1, i + 1)$

```
5     i += 1; y += 2;  
6 }  
7 assert(y == 2 * x);  
8 }
```

Inv is the loop invariant

```
1 void main(int x) {  
2     assume(x >= 0);  
3     int i = 0; int y = 0;  
4     assert(Inv(x, y, i));  
5  
6     y = *; i = *;  
7     assume(Inv(x, y, i));  
8     if (i < x) { // Unrolled  
9         i += 1; y += 2;  
10        assert(Inv(x, y, i));  
11        assume(false);  
12    }  
13  
14    assert(y == 2 * x);  
15 }
```

Specifying Proof Rules in Source Code

As a first step, let's see what it would look like to specify proof rules at the level of source code. As a **toy example** (to save time) we will specify the proof rules for a simple loop invariant inference.

Base Case: $x \geq 0 \Rightarrow \text{Inv}(x, 0, 0)$

```
1 void main(int x) {
```

Inductive Case: $\text{Inv}(x, y, i) \wedge (i < x)$
 $\Rightarrow \text{Inv}(x, y + 1, i + 1)$

```
5     i += 1; y += 2;
```

Adequacy: $\text{Inv}(x, y, i) \wedge (i \geq x)$
 $\Rightarrow y = 2 * x$

Inv is the loop
invariant

```
1 void main(int x) {  
2     assume(x >= 0);  
3     int i = 0; int y = 0;  
4     assert(Inv(x, y, i));  
5  
6     y = *; i = *;  
7     assume(Inv(x, y, i));  
8     if (i < x) { // Unrolled  
9         i += 1; y += 2;  
10        assert(Inv(x, y, i));  
11        assume(false);  
12    }  
13  
14    assert(y == 2 * x);  
15 }
```

Specifying Proof Rules in Source Code

As a first step, let's see what it would look like to specify proof rules at the level of source code. As a **toy example** (to save time) we will specify the proof rules for a simple loop invariant inference.

Base Case: $x \geq 0 \Rightarrow \text{Inv}(x, 0, 0)$

```
1 void main(int x) {
```

Inductive Case: $\text{Inv}(x, y, i) \wedge (i < x)$
 $\Rightarrow \text{Inv}(x, y + 1, i + 1)$

```
5     i += 1; y += 2;
```

Adequacy: $\text{Inv}(x, y, i) \wedge (i \geq x)$
 $\Rightarrow y = 2 * x$

Inv is the loop
invariant

```
1 void main(int x) {  
2     assume(x >= 0);  
3     int i = 0; int y = 0;  
4     assert(Inv(x, y, i));  
5  
6     y = *; i = *;  
7     assume(Inv(x, y, i));  
8     if (i < x) { // Unrolled  
9         i += 1; y += 2;  
10        assert(Inv(x, y, i));  
11        assume(false);  
12    }  
13  
14    assert(y == 2 * x);  
15 }
```



Proof Obligations as Invariant Synthesis

It is well known that invariant inference reduces to program synthesis. However, the problem described on the previous slide does not fit well within any of the existing frameworks for program synthesis. The goal of this project is to characterize and formally study this synthesis problem.

Some existing solutions which do not fit, include...

Specification
Synthesis

SyGuS

SemGuS

“Sketching”



Proof Obligations as Invariant Synthesis

It is well known that invariant inference reduces to program synthesis. However, the problem described on the previous slide does not fit well within any of the existing frameworks for program synthesis. The goal of this project is to characterize and formally study this synthesis problem.

Some existing solutions which do not fit, include...

Specification
Synthesis

Any solution works.

SyGuS

SemGuS

“Sketching”



Proof Obligations as Invariant Synthesis

It is well known that invariant inference reduces to program synthesis. However, the problem described on the previous slide does not fit well within any of the existing frameworks for program synthesis. The goal of this project is to characterize and formally study this synthesis problem.

Some existing solutions which do not fit, include...

Specification
Synthesis

Any solution works.

SyGuS

SemGuS

“Sketching”

Verification is easier than general synthesis.



Proof Obligations as Invariant Synthesis

It is well known that invariant inference reduces to program synthesis. However, the problem described on the previous slide does not fit well within any of the existing frameworks for program synthesis. The goal of this project is to characterize and formally study this synthesis problem.

Some existing solutions which do not fit, include...

Specific

A qualitative comparison can be found in the full paper.

Any solution works.

Verification is easier than general synthesis.

Formalizing the Synthesis Problem

Inductive Predicate Synthesis Modulo Programs



An Overview of the IPS-MP Language

We first define a language which is sufficient for specifying compositional proof rules. The features of the language build on the loop example.



An Overview of the IPS-MP Language

We first define a language which is sufficient for specifying compositional proof rules. The features of the language build on the loop example.

Imperative
Verification Language
(*Standard Features*)

We begin with a standard imperative verification language. This includes **loops**, **function calls**, sources of **non-determinism**, **assumptions**, and **assertions**.



An Overview of the IPS-MP Language

We first define a language which is sufficient for specifying compositional proof rules. The features of the language build on the loop example.

Imperative Verification Language (<i>Standard Features</i>)	We begin with a standard imperative verification language. This includes loops , function calls , sources of non-determinism , assumptions , and assertions .
Partial Predicates (<i>IPS-MP Features</i>)	We then introduce support for partial predicates. These are pure Boolean functions with one or more holes in the implementation. These functions may only appear in assume or assert statements.



The Solution to an IPS-MP Problem

The solution to an IPS-MP problem is an implementation for each predicate, which can be obtained by filling each hole with a Boolean expression, such that the new program is correct with respect to all of its assertions.

IPS-MP: Inductive Predicate Synthesis Modulo Programs



The Solution to an IPS-MP Problem

The solution to an IPS-MP problem is an implementation for each predicate, which can be obtained by filling each hole with a Boolean expression, such that the new program is correct with respect to all of its assertions.

No Partial Predicates

Interprocedural program
verification.

Partial Predicates
Without Assertions

Specification synthesis
without biases (trivial).

General IPS-MP

A form of adequate proof-rule
synthesis.

IPS-MP: Inductive Predicate Synthesis Modulo Programs



The Solution to an IPS-MP Problem

The solution to an IPS-MP problem is an **implementation for each predicate**, which can be obtained by filling each hole with a Boolean expression, such that the new program is correct with respect to all of its assertions.

No Partial Predicates

Interprocedural program
verification.

Partial Predicates
Without Assertions

Specification synthesis
without biases (trivial).

General IPS-MP

A form of adequate proof-rule
synthesis.

IPS-MP: **Inductive Predicate Synthesis** Modulo Programs



The Solution to an IPS-MP Problem

The solution to an IPS-MP problem is an implementation for each predicate, which can be obtained by filling each hole with a Boolean expression, **such that the new program is correct** with respect to all of its assertions.

No Partial Predicates

Interprocedural program
verification.

Partial Predicates Without Assertions

Specification synthesis
without biases (trivial).

General IPS-MP

A form of adequate proof-rule
synthesis.

IPS-MP: Inductive Predicate Synthesis **Modulo Programs**



Formalizing Our Toy Example as an IPS-MP Problem

```
bool Inv(int x, int y, int i) {  
    return synth(x, i);  
}  
  
void main(int x) {  
    assume(x >= 0);  
    int i = 0; int y = 0;  
    assert(Inv(x, y, i));  
    y = *; i = *;  
    assume(Inv(x, y, i));  
    if (i < x) {  
        i += 1; y += 2;  
        assert(Inv(x, y, i));  
        assume(false);  
    }  
    assert(y == 2 * x);  
}
```



Formalizing Our Toy Example as an IPS-MP Problem

```
bool Inv(int x, int y, int i) {  
    return synth(x, i);  
}  
void main(int x) {  
    assume(x >= 0);  
    int i = 0; int y = 0;  
    assert(Inv(x, y, i));  
    y = *; i = *;  
    assume(Inv(x, y, i));  
    if (i < x) {  
        i += 1; y += 2;  
        assert(Inv(x, y, i));  
        assume(false);  
    }  
    assert(y == 2 * x);  
}
```

} Partial Predicate

Formalizing Our Toy Example as an IPS-MP Problem

```
bool Inv(int x, int y, int i) {  
    return synth(x, i);  
}  
void main(int x) {  
    assume(x >= 0);  
    int i = 0; int y = 0;  
    assert(Inv(x, y, i));  
    y = *; i = *;  
    assume(Inv(x, y, i));  
    if (i < x) {  
        i += 1; y += 2;  
        assert(Inv(x, y, i));  
        assume(false);  
    }  
    assert(y == 2 * x);  
}
```

Partial Predicate

Hole

Formalizing Our Toy Example as an IPS-MP Problem

```
bool Inv(int x, int y, int i) {  
    return synth(x, i);  
}  
void main(int x) {  
    assume(x >= 0);  
    int i = 0; int y = 0;  
    assert(Inv(x, y, i));  
    y = *; i = *;  
    assume(Inv(x, y, i));  
    if (i < x) {  
        i += 1; y += 2;  
        assert(Inv(x, y, i));  
        assume(false);  
    }  
    assert(y == 2 * x);  
}
```

Partial Predicate

Hole

One solution to this IPS-MP problem is to synthesize the Boolean expression,

return `y == 2 * i && i <= x;`

as an implementation for the predicate `Inv(x, y, i)`. In this case, the original program is correct.

Formalizing Our Toy Example as an IPS-MP Problem

```
bool Inv(int x, int y, int i) {  
    return synth(x, i);  
}
```

} Partial Predicate

```
void main(int x) {
```

```
    assume(x >=
```

```
    int i = 0
```

```
    ass
```

```
    y
```

```
    assum
```

```
    if (
```

```
        i += 1; y +
```

```
        assert(Inv(x, y, i)),
```

```
        assume(false);
```

```
    }
```

```
    assert(y == 2 * x);
```

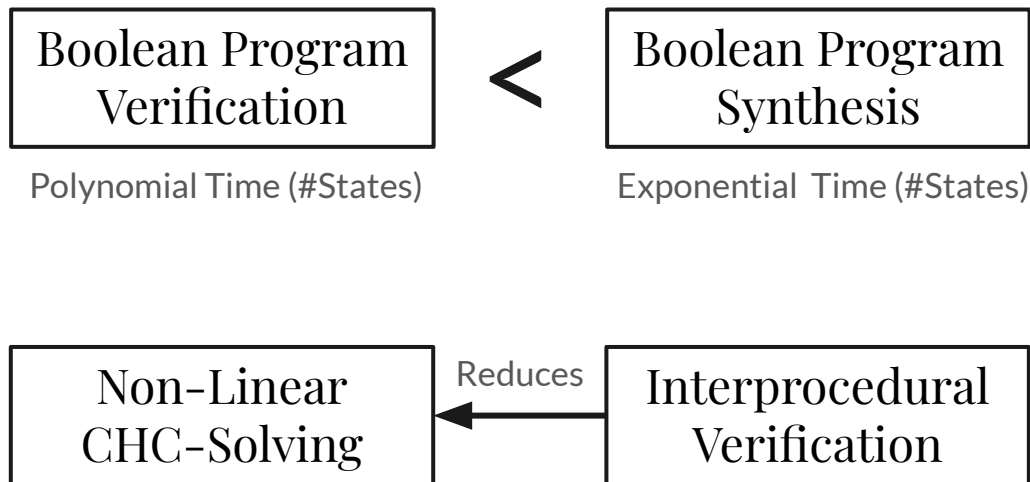
```
}
```

A real IPS-MP solver could verify the program on the left without any instrumentation... IPS-MP is a strict generalization of inductive program verification.

as an implementation for the predicate $\text{Inv}(x, y, i)$. In this case, the original program is correct.

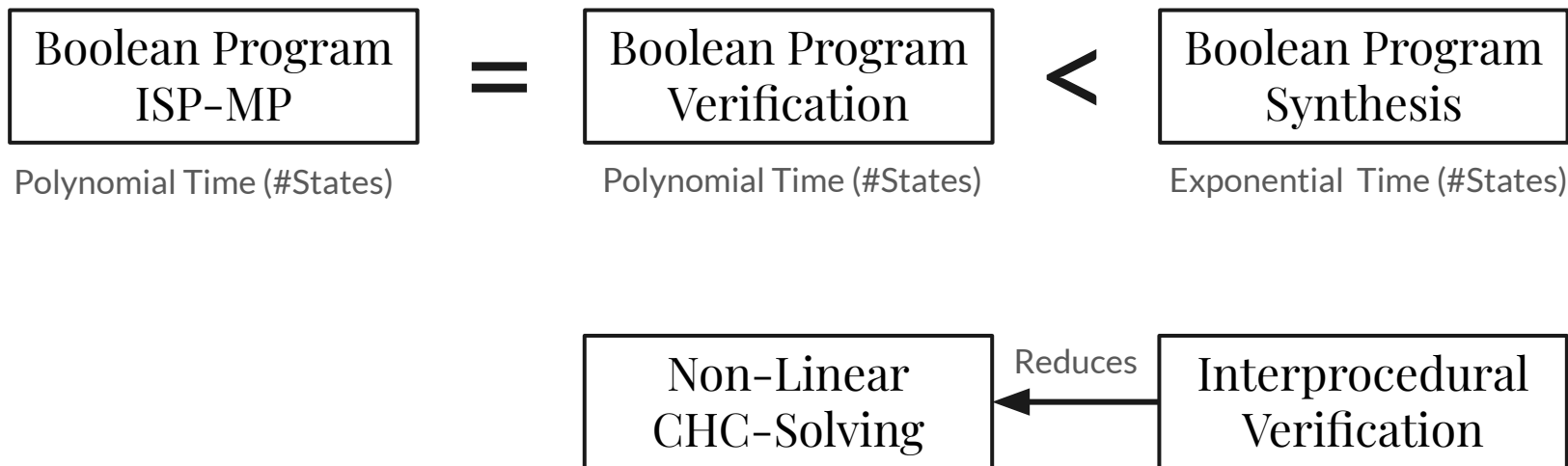


The Cost of IPS-MP



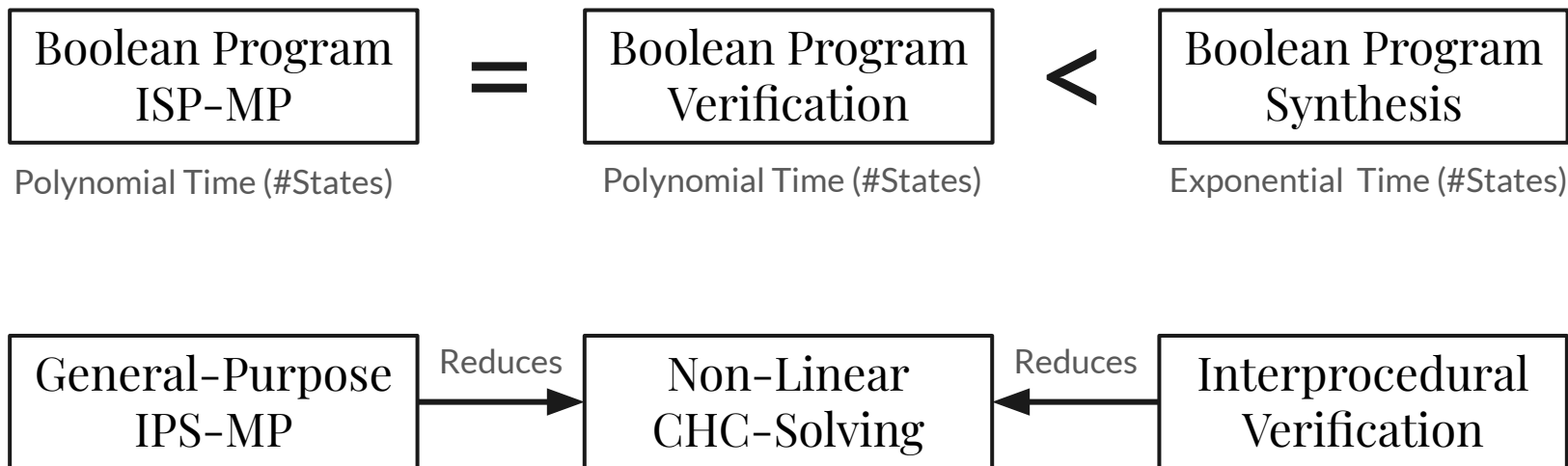


The Cost of IPS-MP





The Cost of IPS-MP



IPS-MP In Practice

Instantiating Proof Rules in SeaHorn

We Have Implemented IPS-MP Support in SeaHorn!



github.com/seahorn/seahorn

With SeaHorn IPS-MP, we have realized:

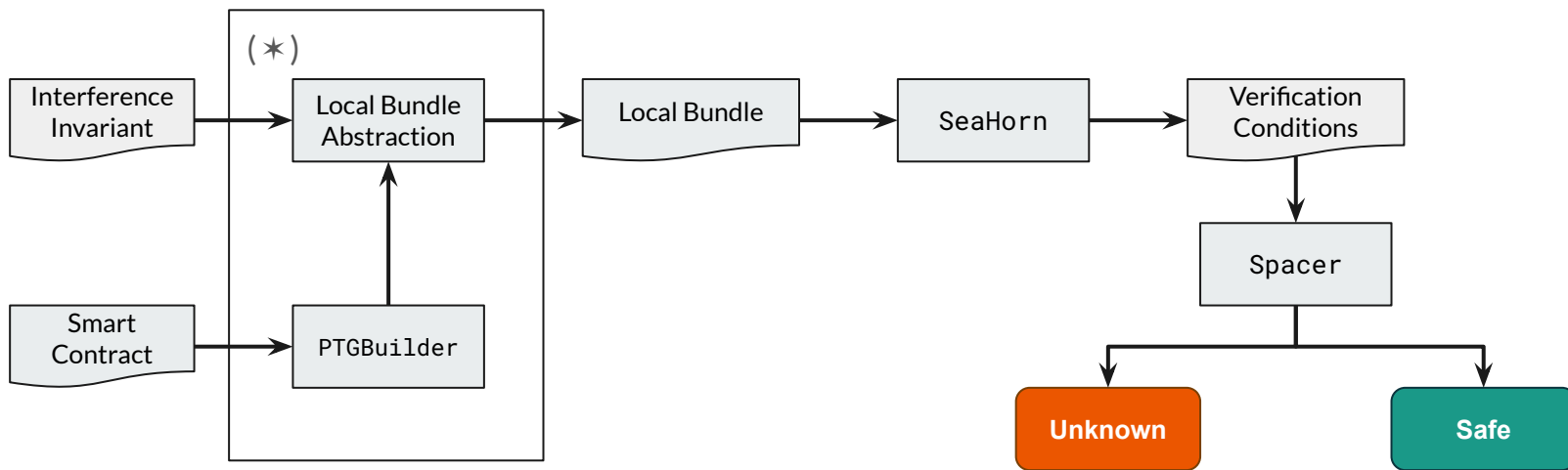
Class Invariant Inference

Compositional Array
Abstractions (**PCMC**)

Parameterized Ring
Analysis (**PCMC**)

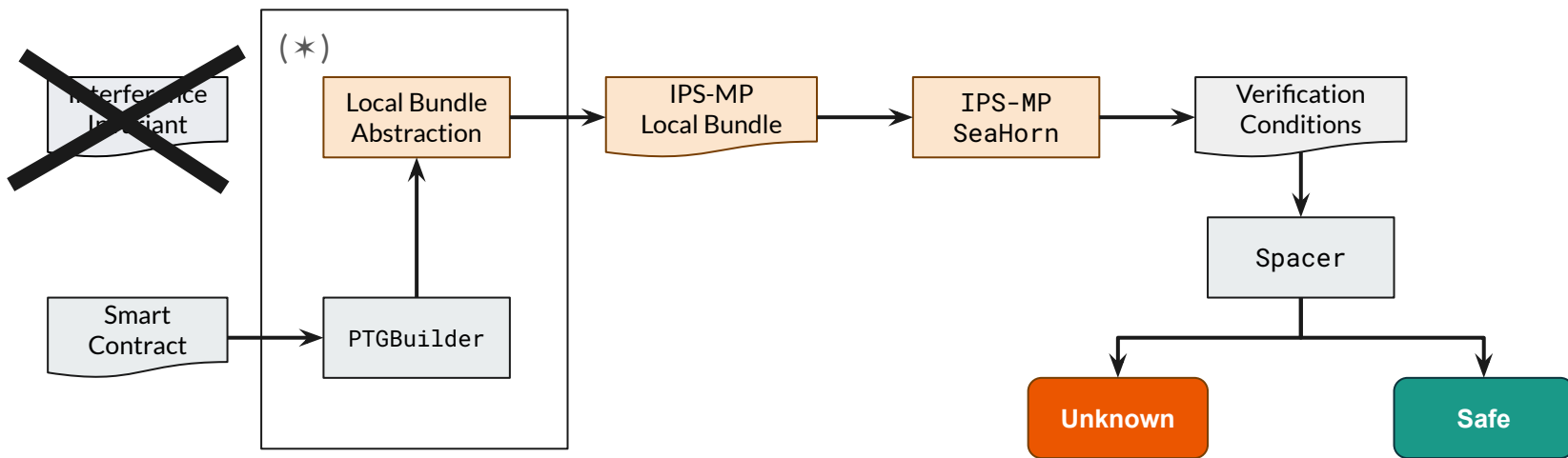
User-Guided Loop
Invariant Inference

Back to the Motivating Example



(*) *Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE (2022)*. S. Wesley, M. Christakis, J. A. Navas, R. J. Trefer, V. Wüstholtz, A. Gurfinkel. In VMCAI 2022.

Back to the Motivating Example



(*) *Verifying Solidity Smart Contracts via Communication Abstraction in SmartACE (2022).* S. Wesley, M. Christakis, J. A. Navas, R. J. Trefer, V. Wüstholtz, A. Gurfinkel. In VMCAI 2022.

Empirical Results

And Directions for Future Work

Do Existing Synthesizers Perform Well on IPS-MP?

Benchmarks				IPS-MP (SPACER)			IPS-MP (ELDARICA)				HORNSpec			CVC4		
Type	Safe	Buggy	Preds	Time	TO	✓	Time	TO	MEM	✓	Time	UN	✓	TO	N/A	✓
Loop	7	7	9	4	0	14	45	0	0	14	4	12	2	7	7	0
Class	6	6	6	2	0	12	1 449	0	0	12	—	12	0	6	6	0
Array	4	6	6	4	0	10	230	0	0	10	—	10	0	4	6	0
Ring	2	3	2	1	0	5	52	0	0	5	—	5	0	2	3	0
Proc	3	3	3	2	0	6	4	0	0	6	—	6	0	3	3	0
SC	70	4	181	6 878	4	70	6 717	53	12	9	—	—	—	—	—	—
Total	92	29	207	6 891	4	117	8 497	53	12	56	4	45	2	22	29	0

No: existing synthesizer solvers struggle with IPS-MP instances. This suggests the need for optimized reductions to standard synthesis techniques, or specialized decision procedures for IPS-MP.



Does IPS-MP SmartACE Still Perform Well?

Contracts				Performance (Time)		
Name	Props	Preds	✓	VERX [52]	SMARTACE (Manual) [66]	SMARTACE (IPS-MP)
Alchemist	3	12	3	29	7	208
Brickblock	6	12	6	191	13	1214
Crowdsale	9	27	9	261	223	238
ERC20	9	27	9	158	12	103
Melon	16	32	16	408	30	979
PolicyPal	4	16	4	20 773	26	3118
VUToken	5	22	1	715	19	17
Zebi	5	14	5	77	8	487
Zilliqa	5	10	5	94	8	501
Total	62	172	58	22 706	346	5685

Yes: SmartACE (IPS-MP) incurs several magnitudes of overhead compared to SmartACE, but performs comparably to VerX (*a semi-automated solution, which relies on manual predicate abstraction*).

Summary of Contributions

1. We identify a **new synthesis problem (IPS-MP)** with many applications to the design of compositional program analysis tools.
2. We establish that **IPS-MP is strictly easier** than general synthesis.
3. We show that **IPS-MP reduces to non-linear CHC-solving**, and use this to implement an IPS-MP solver in SeaHorn.
4. We **establish a set of benchmarks** to evaluate future IPS-MP solvers, and show that IPS-MP can be used to **automate an existing tool (SmartACE)**.